# Project 2 – Acme Analytics Linux Platform

## Day 6 · Phase 3: systemd units revised, custom target, a look inside server mode

Muhammad Bilal Javed · 21 October 2026 · study copy

| | |
|---|---|
| **Project** | Acme Analytics – hardened, observable and recoverable Linux platform |
| **Phase** | 3 – Automation |
| **Hosts worked on** | stor01 |
| **Status** | Phase 3 🟡 – Steps 3.1–3.3 ✅ (Day 5) · Step 3.4 custom target ✅ · Step 3.5 cron vs timer and nightly backup next |
| **Author** | Muhammad Bilal Javed |

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 5.

---

## 1. Revision – what a systemd unit really is

💡 **In plain words:** before going on, I explained Steps 3.2 and 3.3 again in my own words and corrected what I had misunderstood.

| I thought | Correct |
|---|---|
| A unit is a script | A unit is a **settings file**; it **points** to the program or script with `ExecStart=` |
| Everything in Linux is a unit | Only what **systemd manages**: services, timers, sockets, targets, mounts … – `/etc/hosts` is not a unit |
| All units are in `/etc/systemd/system/` | Linux's own units: `/lib/systemd/system/` · **my** units: `/etc/systemd/system/` |
| The `.timer` file must name its service | The link is the **same first name**: `disk-alert.timer` → `disk-alert.service`, `hello.socket` → `hello@.service` |
| `WantedBy=timers.target` links timer and service | It only makes the timer start **at every boot** (after `enable`) |
| `multi-user.target` = things related to users | The normal **server state**: networking and services, text login, no desktop; `graphical.target` = that + desktop |
| `nc` = network card | **netcat** – a small client tool that connects to a port |

```
TIMER:   disk-alert.timer ──same name──► disk-alert.service ──ExecStart──► disk-alert.sh ──logger──► journal
           WHEN                              HOW                                WHAT
SOCKET:  client connects :9999 ──► hello.socket ──same name──► one copy of hello@.service ──► answer to client
──► copy ends
```

**`Accept=yes` vs `Accept=no`:** like a **call centre** – every caller gets their own new agent (one copy of `hello@.service` per connection) – versus a **receptionist** – the first caller wakes up **one** manager (`ssh.service`) who then handles everyone.

### Key understanding (in my own words)

**systemd** is the program that starts and manages services on a Linux server. Everything it manages (SSH, Docker, my disk alert) is a **unit**: a small settings file that says how something should run. Linux's own units are in `/lib/systemd/system/` (for example `ssh.service` and `ssh.socket`), and the units I create for automation go in `/etc/systemd/system/`. The ending of the name says the type: **`.service`** = **how** to run a program, **`.timer`** = **when** to run it, **`.socket`** = **which port** to listen on, and **`.target`** = a **group** of units that start together. After creating or changing a unit I run `systemctl daemon-reload`, and `enable --now` starts it now and at every boot.

**Timer (Step 3.2):** the script `/usr/local/bin/disk-alert.sh` checks how full `/data` is and writes a warning to the journal only if usage reaches the threshold (80 %). `disk-alert.service` says **how** to run it (`Type=oneshot`, `ExecStart=` the script), and `disk-alert.timer` says **when** – 2 minutes after boot, then every 5 minutes. The timer finds its service **by the same first name** (`disk-alert`); `WantedBy=timers.target` only makes the timer start at every boot. The chain is **timer → service → script → journal**.

**Socket (Step 3.3):** here the trigger is a **connection**, not a time. A rarely used service does not need to run all day, so **systemd itself listens on the port** and starts the program only when someone connects. `hello.socket` says `ListenStream=9999` and finds its service by the same first name, `hello`. With **`Accept=yes`**, every connection gets **its own short-lived copy** of the service – that is why the service is a **template**, `hello@.service`: the `@` means one file, many copies, one per connection. `ExecStart=/bin/sh -c 'echo …'` is the program, and **`StandardOutput=socket`** sends its output straight back through the connection to the client. I tested it with `nc localhost 9999` and got *"hello from stor01 – started on demand"*; the copy ended, so nothing was running before or after. The chain is **client connects → socket → one copy of the service → answer → copy ends**. Real servers use this too: with `ssh.socket`, systemd holds port 22 and starts `ssh.service` on the first connection (with `Accept=no`, one sshd handles all connections itself – no template). On stor01, `ssh.socket` is disabled and `ssh.service` runs all the time – the classic choice for a server admins use daily. My three lessons: a per-connection service needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit before I use it.

## 2. Step 3.4 – a custom target: one command for the whole Acme stack

💡 **The story:** stor01 now has several Acme units of my own – the disk alert, the hello socket, later the nightly backup. During maintenance an admin should not have to remember every name. One group, **`acme-stack.target`**, starts them all with one command – the same idea as `multi-user.target`, but for Acme.

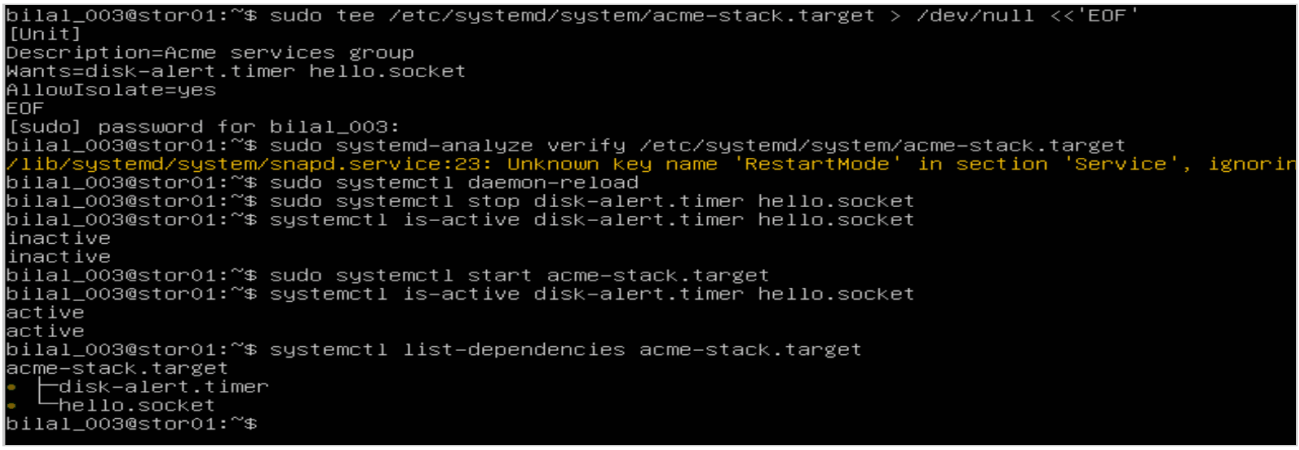
All on **stor01**.

```
        sudo tee /etc/systemd/system/acme-stack.target > /dev/null <<'EOF'
        [Unit]
        Description=Acme services group
        Wants=disk-alert.timer hello.socket
        AllowIsolate=yes
        EOF
        sudo systemd-analyze verify /etc/systemd/system/acme-stack.target
        sudo systemctl daemon-reload

        # test: stop both members, then start the whole group with ONE command
        sudo systemctl stop disk-alert.timer hello.socket
        systemctl is-active disk-alert.timer hello.socket      # inactive inactive
        sudo systemctl start acme-stack.target
        systemctl is-active disk-alert.timer hello.socket      # active active
        systemctl list-dependencies acme-stack.target          # the group as a tree
```

| Setting | Meaning |
|---|---|
| `Wants=` | The members of the group – started when the target starts |
| `AllowIsolate=yes` | The server may later be switched into this target like a “mode” |

📸 **Evidence – both members stopped, one `start` of the target, both active again; the group tree (the yellow line is Ubuntu’s own `snapd.service`, not my file):**




*acme-stack target*

### Key understanding (in my own words)

A **target** is a unit that **groups other units** – services, timers or sockets – so that starting the target starts all its members with one command. Linux’s own targets are in `/lib/systemd/system/`: `timers.target` holds all the timers (that is why each timer has `WantedBy=timers.target`), and `multi-user.target` is the normal **server state** – networking, SSH and all other services, with text login and no desktop. `graphical.target` is `multi-user.target` plus the graphical desktop, so targets can contain other targets; my servers boot to `multi-user.target`. On stor01 I created my own group, `acme-stack.target`, and `Wants=disk-alert.timer hello.socket` made both units its members. To test it, I stopped both (`inactive inactive`), started only `acme-stack.target`, and both became `active` again; `systemctl list-dependencies acme-stack.target` showed the group as a tree. In a real company this is how an admin brings a whole application stack back after maintenance with one command – for example `company-app.target` – instead of remembering every service name.

## 3. Reference – looking inside the real server

💡 **In plain words:** to see that these ideas are not only lab examples, I looked at how Ubuntu itself uses sockets, targets and timers on stor01.

```
        # stor01
        systemctl list-sockets                                  # 29 sockets held by systemd — hello.socket on
[::]:9999
        sudo systemctl status ssh.socket ssh.service            # socket mode or always-on?
        systemctl list-dependencies multi-user.target --no-pager    # what "server mode" contains
        ls /etc/systemd/system/multi-user.target.wants/         # members I or packages enabled
        ls /etc/systemd/system/timers.target.wants/
        ls /etc/systemd/system/sockets.target.wants/
```

📸 **Evidence – `ssh.socket` disabled and inactive, `ssh.service` enabled and running since boot (5.2 MB); `ExecStartPre=/usr/sbin/sshd -t` tests the configuration before starting:**

```
bilal_003@stor01:~$ sudo systemctl status ssh.socket ssh.service
[sudo] password for bilal_003:
● ssh.socket - OpenBSD Secure Shell server socket
     Loaded: loaded (/lib/systemd/system/ssh.socket; disabled; vendor preset: enabled)
     Active: inactive (dead)
     Listen: [::]:22 (Stream)
   Accepted: 0; Connected: 0;

● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-10-21 16:40:15 UTC; 36min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1031 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1067 (sshd)
      Tasks: 1 (limit: 964)
     Memory: 5.2M
        CPU: 1.939s
     CGroup: /system.slice/ssh.service
             └─1067 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

*ssh socket vs service*

📸 **Evidence – `multi-user.target` contains other targets: `timers.target` with my `disk-alert.timer` next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`), `getty.target` with the console login `getty@tty1.service` (a template again), and `nfs-client.target` / `remote-fs.target`:**

```
bilal_003@stor01:~$ systemctl list-dependencies multi-user.target --no-pager | tail -35
●   ├─apport-autoreport.timer
●   ├─apt-daily-upgrade.timer
●   ├─apt-daily.timer
●   ├─disk-alert.timer
●   ├─dpkg-db-backup.timer
●   ├─e2scrub_all.timer
●   ├─fstrim.timer
●   ├─fwupd-refresh.timer
●   ├─logrotate.timer
●   ├─man-db.timer
●   ├─motd-news.timer
●   ├─snapd.snap-repair.timer
●   ├─systemd-tmpfiles-clean.timer
●   ├─ua-timer.timer
●   ├─update-notifier-download.timer
●   └─update-notifier-motd.timer
● ├─cloud-init.target
●   ├─cloud-config.service
●   ├─cloud-final.service
●   ├─cloud-init-local.service
●   └─cloud-init.service
● ├─getty.target
●   ├─getty-static.service
●   └─getty@tty1.service
● ├─nfs-client.target
●   ├─auth-rpcgss-module.service
●   ├─nfs-blkmap.service
●   ├─rpc-statd-notify.service
●   └─remote-fs-pre.target
● └─remote-fs.target
●   └─nfs-client.target
●     ├─auth-rpcgss-module.service
●     ├─nfs-blkmap.service
●     ├─rpc-statd-notify.service
●     └─remote-fs-pre.target
bilal_003@stor01:~$
```

*multi-user tree*

📸 **Evidence – `enable` = a link in a `….wants/` folder: `ssh.service`, `nfs-server.service`, `glpi-agent.service` in `multi-user.target.wants/`; `disk-alert.timer` in `timers.target.wants/`; `hello.socket` in `sockets.target.wants/`:**

```
bilal_003@stor01:~$ ls /etc/systemd/system/multi-user.target.wants/
console-setup.service         pollinate.service                       snap-lxd-29351.mount
cron.service                  remote-fs.target                        snap-lxd-40911.mount
dmesg.service                 rpcbind.service                         snap.lxd.activate.service
e2scrub_reap.service          rsyslog.service                         snap-snapd-21759.mount
glpi-agent.service            secureboot-db.service                   snap-snapd-28254.mount
grub-common.service           snap-core20-2318.mount                  ssh.service
grub-initrd-fallback.service  snap-core20-2922.mount                  systemd-networkd.service
irqbalance.service            snap-core22-2955.mount                  systemd-resolved.service
lxd-agent.service             snapd.apparmor.service                  thermald.service
ModemManager.service          snapd.autoimport.service                ua-reboot-cmds.service
networkd-dispatcher.service   snapd.core-fixup.service               ubuntu-advantage.service
nfs-client.target             snapd.recovery-chooser-trigger.service  ufw.service
nfs-server.service            snapd.seeded.service                    unattended-upgrades.service
open-vm-tools.service         snapd.service
bilal_003@stor01:~$ ls /etc/systemd/system/timers.target.wants/
apport-autoreport.timer  e2scrub_all.timer    motd-news.timer
apt-daily.timer          fstrim.timer         snapd.snap-repair.timer
apt-daily-upgrade.timer  fwupd-refresh.timer  ua-timer.timer
disk-alert.timer         logrotate.timer      update-notifier-download.timer
dpkg-db-backup.timer     man-db.timer         update-notifier-motd.timer
bilal_003@stor01:~$ ls /etc/systemd/system/sockets.target.wants/
apport-forward.socket  iscsid.socket      snapd.socket                        systemd-networkd.socket
dm-event.socket        multipathd.socket  snap.lxd.daemon.unix.socket         uuidd.socket
hello.socket           rpcbind.socket     snap.lxd.user-daemon.unix.socket
bilal_003@stor01:~$
```

*wants folders*

| What I saw | Meaning |
|---|---|
| `list-sockets`: 29 sockets, `hello.socket` → ACTIVATES – | systemd holds many ports and paths for Linux itself; – because a new copy of the template starts per connection |
| `ssh.socket` disabled, `ssh.service` running | stor01 uses classic always-on SSH – normal for a server admins use daily |
| `ExecStartPre=/usr/sbin/sshd -t` | Test the config **before** start – like `bash -n` and `systemd-analyze verify` |
| `Docs: man:sshd(8)` | The manual pages, not the configuration (that is `/etc/ssh/sshd_config`) |
| `.mount` units in the tree | Mounts are units too – `/data` and `/backups` from fstab become `data.mount`, `backups.mount` |
| ● / other symbol | Active now / not running now – often a one-time job already finished at boot |
| `….wants/` folders | `enable` creates a link there – that link is what makes a unit a member of a target |

### Key understanding (in my own words)

> `systemctl list-dependencies multi-user.target` showed me what "server mode" really is: a tree of units. Its direct members are services like `cron`, `rsyslog`, `nfs-server`, `rpcbind`, `ssh`, `glpi-agent` and `open-vm-tools`, plus `.mount` units – mounts are units too. It also contains **other targets**: `timers.target`, where my `disk-alert.timer` sits next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`); `sockets.target` with `hello.socket`; `getty.target`, whose `getty@tty1.service` is the console login and also a template with `@`; and `remote-fs.target` for NFS. The `….wants/` folders in `/etc/systemd/system/` show how membership works: `enable` simply creates a link in the folder of the target named in `WantedBy=`. On stor01, `ssh.socket` is disabled and `ssh.service` always runs, and its `ExecStartPre=sshd -t` tests the configuration before every start. So a target is a group, groups can contain groups, `enable` adds a member, and booting the server simply means systemd starting this whole tree.

## 4. Next steps

1. **Step 3.5 part A:** cron, the old way – run the disk check from `crontab` for a few minutes, read the cron line field by field, then remove it.
2. **Step 3.5 part B:** the **nightly backup** of `/data/app` as a service + timer at 02:00 with `Persistent=true` (catches up after downtime), keeping 7 days – and cron vs timer compared.
3. Phase 2 tickets in GLPI: LVM build, grow /data, fstab UUID typo, NFS share, stale NFS mount after rollback.